Power BI project

In this gym club system, each trainee is assigned to a specific trainer. Trainers create personalized workout programs for their trainees. Trainees log their progress by marking completed exercises and then wait for feedback from their trainer. Once feedback is received, the trainee adjusts their program based on the trainer's recommendations and continues their training journey until they achieve their fitness goals.

Tables:

1. User: Stores general information for all users (athletes, coaches, and admins).

Columns:

id	Primary key
Full_name	User's full name
Phone_number	User's phone number
Email	User's email address (used for login)
password	Hashed password for login
role	Admin, coach or athlete
Is_active	Active or inactive
Created_at	Registration date

2. AthleteProfile: Stores specialized information and fitness goals for athletes.

Columns:

User_id	User id
age	Athlete's age
Weight	Athlete's weight
height	Athlete's height
physical_limits	Movement restrictions
Goal_type	Goal type (e.g., weight loss)
initial_value	Starting value for the goal
current_value	Current progress value
target_value	Final target value
updated_at	Last update date

3. Excersise: Stores all available exercises for easy reference.

Columns:

id	Primary key
name	Exercise name
description	Exercise description

4. WorkoutPlan: Stores training program information.

Columns:

id	Primary key
Coach_id	Coach id
Athlete_id	User id
Created_at	Plan creation date
Start_date	Program start date
End_date	Program end date
status	Plan status: Active or Completed

5. PlanDays: Stores session structure within each workout plan. Determines how many different sessions make up a plan and their order.

Columns:

id	Primary key
Plan_id	Plan id
Day_name	Name of the day
Day_indx	Session order (Day 1, then Day 2, etc.)

6. PlanExercises: Stores exercise details for each day in the plan (exercise name, sets, reps, etc.).

Columns:

id	Primary key
Day_id	Day id
Exercise_id	Exercise id
Sets_planned	Number of sets prescribed by coach
reps_planned	Number of repeats prescribed by coach
Rest_seconds	Rest time between sets (in seconds)

7. WorkoutLogs: Stores the athlete's logged progress on their workout program.

Columns:

id	Primary key
Athlete_id	User id
Plan_Exercises_id	Plan exercise id
Performed_date	Date exercise was performed
Set_number	Set number
Reps_done	Number of repeats completed
Is_completed	1 = Completed, 0 = Not completed

8. Feedbacks: Stores feedback from coaches to their athletes.

Columns:

id	Primary key
plan_id	Plan id
content	Feedback message
Send_date	Date feedback was sent
Is_read	Whether the athlete has read it

9. Ratings: Stores athlete ratings for their coach.

Columns:

id	Primary key
plan_id	Plan id
score	Rating score (0 to 5)
Created_at	Rating submission date

10. AthleteProfile_History: Stores historical progress records for athletes.

Columns:

id	Primary key
User_id	User id
initial_value	Starting value for this record
current_value	Current progress value
target_value	Final target value
updated_at	Record update date

Then, according to the tables above, we generate data.

Data:

User:

id	full_name	phone_number	email	password	role	is_active	created_at
1	admin	9111251252	admin@gym.com	sthadmin	admin	1	2026-01-01
2	user1	9111251253	user1@example.com	sth1	coach	1	2025-11-15
3	user2	9111251254	user2@example.com	sth2	coach	1	2025-10-01
4	user3	9111251255	user3@example.com	sth3	coach	1	2025-12-01
5	user4	9111251256	user4@example.com	sth4	coach	1	2025-09-15
6	user5	9111251257	user5@example.com	sth5	athlete	1	2026-01-10
7	user6	9111251258	user6@example.com	sth6	athlete	1	2025-12-20
8	user7	9111251259	user7@example.com	sth7	athlete	1	2026-02-01
9	user8	9111251260	user8@example.com	sth8	athlete	1	2025-12-15
10	user9	9111251261	user9@example.com	sth9	athlete	1	2026-01-20
11	user10	9111251262	user10@example.com	sth10	athlete	1	2026-02-10
12	user11	9111251263	user11@example.com	sth11	athlete	1	2026-02-15
13	user12	9111251264	user12@example.com	sth12	athlete	1	2025-11-01
14	user13	9111251265	user13@example.com	sth13	athlete	1	2025-12-10
15	user14	9111251266	user14@example.com	sth14	athlete	1	2026-01-05
16	user15	9111251267	user15@example.com	sth15	athlete	1	2026-03-01
17	user16	9111251268	user16@example.com	sth16	athlete	1	2025-12-01

AthleteProfile:

	A	B	C	D	E	F	G	H	I	J
1	user_id	age	weight	height	physical_limits	goal_type	initial_value	current_value	target_value	updated_at
2	6	28	85.5	178	None	Weight Loss	92	85.5	75	2026-07-10
3	7	24	60	165	Knee Pain	Strength Gain	50	65	80	2026-07-10
4	8	32	95	182	None	Weight Loss	0	0	0	2026-07-10
5	9	27	70	170	None	Weight Loss	78	70	60	2026-07-10
6	10	29	80	175	Back Pain	Strength Gain	60	75	90	2026-07-10
7	11	22	58	160	None	Weight Loss	65	58	50	2026-07-10
8	12	35	88	180	None	Strength Gain	80	88	95	2026-07-10
9	13	26	65	168	None	Weight Loss	0	0	0	2026-07-10
10	14	40	75	175	None	Weight Loss	85	75	70	2026-07-10
11	15	31	72	172	None	Strength Gain	65	72	85	2026-07-10
12	16	29	85	180	None	Weight Loss	90	85	75	2026-07-10
13	17	26	63	165	None	Strength Gain	55	63	75	2026-07-10

For weight_loss, initial_value represents the athlete's starting body weight. For strength_gain, initial_value represents the starting weight of the barbell or dumbbell used

Excersise:

	A	B	C
1	id	name	description
2	1	Barbell Squat	Lower body movement for quads and glutes
3	2	Barbell Bench Press	Upper body movement for chest and shoulders
4	3	Deadlift	Full body movement for back and hamstrings
5	4	Pull-Ups	Back movement for lats
6	5	Shoulder Press	Shoulder and arm movement
7	6	Leg Press	Lower body machine movement
8	7	Lat Pulldown	Back machine movement
9	8	Dumbbell Curl	Bicep movement
10	9	Tricep Pushdown	Tricep movement
11	10	Plank	Core and midsection movement
12			

WorkoutPlan:

	A	B	C	D	E	F	G
1	id	coach_id	athlete_id	created_at	start_date	end_date	status
2	1	2	6	2026-05-11	2026-05-11	2026-06-10	completed
3	2	2	6	2026-06-15	2026-06-15	2026-07-15	active
4	3	2	7	2026-05-26	2026-05-26	2026-06-25	completed
5	4	2	7	2026-07-01	2026-07-01	2026-07-30	active
6	5	2	8	2026-06-20	2026-06-20	2026-07-20	active
7	6	2	9	2026-05-21	2026-05-21	2026-06-20	completed
8	7	2	9	2026-06-25	2026-06-25	2026-07-25	active
9	8	3	10	2026-06-01	2026-06-01	2026-06-30	completed
10	9	3	10	2026-07-05	2026-07-05	2026-08-05	active
11	10	3	11	2026-06-12	2026-06-12	2026-07-02	completed
12	11	3	12	2026-05-16	2026-05-16	2026-06-15	completed
13	12	3	12	2026-06-20	2026-06-20	2026-07-20	active
14	13	3	13	2026-06-25	2026-06-25	2026-07-25	active
15	14	4	14	2026-05-01	2026-05-01	2026-05-31	completed
16	15	4	14	2026-06-10	2026-06-10	2026-07-10	active
17	16	4	15	2026-07-01	2026-07-01	2026-07-31	active
18	17	5	16	2026-06-15	2026-06-15	2026-07-15	active
19	18	5	17	2026-05-20	2026-05-20	2026-06-19	completed
20	19	5	17	2026-06-25	2026-06-25	2026-07-25	active

PlanDays:

	A	B	C	D	E
1	id	plan_id	day_name	day_indx	
2	1	1	Leg Day	1	
3	2	1	Upper Body Day	2	
4	3	2	Heavy Day	1	
5	4	2	Light Day	2	
6	5	3	Day 1	1	
7	6	3	Day 2	2	
8	7	4	Legs & Glutes	1	
9	8	4	Chest & Back	2	
10	9	5	Upper Body	1	
11	10	5	Lower Body	2	
12	11	6	Full Body 1	1	
13	12	6	Full Body 2	2	
14	13	7	Strength	1	
15	14	7	Conditioning	2	
16	15	8	Upper Body	1	
17	16	8	Lower Body	2	
18	17	9	Heavy	1	
19	18	9	Light	2	

For example, in Program 1, the leg day session must be completed first, followed by the upper body day session

PlanExercises:

id	day_id	exercise_id	sets_planned	reps_planned	rest_seconds
1	1	1	4	10	60
2	1	6	3	12	45
3	1	10	3	30	30
4	2	2	4	10	60
5	2	4	3	8	90
6	2	8	3	12	30
7	3	1	5	5	120
8	3	3	4	5	120
9	3	10	3	45	45
10	4	2	3	15	45
11	4	5	3	12	45
12	5	2	4	10	60
13	5	7	3	10	45
14	6	1	4	8	90
15	6	6	3	12	60
16	7	1	4	10	60
17	7	6	3	12	45

AthleteProfile_History:

id	user_id	initial_value	current_value	target_value	updated_at
1	6	92	90	75	2026-01-15
2	6	92	88	75	2026-02-15
3	6	92	86.5	75	2026-03-15
4	6	92	85.5	75	2026-07-10
5	7	50	52	80	2026-01-20
6	7	50	57	80	2026-02-20
7	7	50	61	80	2026-03-20
8	7	50	65	80	2026-07-10
9	8	0	0	0	2026-01-10
10	8	0	0	0	2026-07-10
11	9	78	75	60	2026-01-05
12	9	78	73	60	2026-02-05
13	9	78	70	60	2026-07-10
14	10	60	65	90	2026-01-25
15	10	60	70	90	2026-02-25
16	10	60	75	90	2026-07-10
17	11	65	62	50	2026-01-01

For example, for the first row of Plan 1:

In the leg day session (day_id = 1), the athlete must perform barbell squats (exercise_id = 1), with four planned sets (sets_planned = 4), ten planned reps (reps_planned = 10), and a 60 second rest interval between sets (rest_seconds = 60).

WorkoutLogs:

id	athlete_id	plan_exercises_id	performed_date	set_number	reps_done	is_completed
1	6	1	2026-07-08	1	10	1
2	6	1	2026-07-08	2	10	1
3	6	1	2026-07-08	3	9	1
4	6	1	2026-07-08	4	10	1
5	6	2	2026-07-08	1	12	1
6	6	2	2026-07-08	2	11	1
7	6	2	2026-07-08	3	12	1
8	6	4	2026-07-07	1	10	1
9	6	4	2026-07-07	2	10	1
10	6	4	2026-07-07	3	10	1
11	6	1	2026-07-05	1	10	1
12	6	1	2026-07-05	2	10	1
13	6	1	2026-07-05	3	10	1

For example, for athlete 1:

The coach has planned 4 different sets. The number of reps performed by the user has been entered, and because the set was completed (the user did not abandon the set and went from beginning to end), the status is completed (it doesn't matter if it was performed perfectly or not).

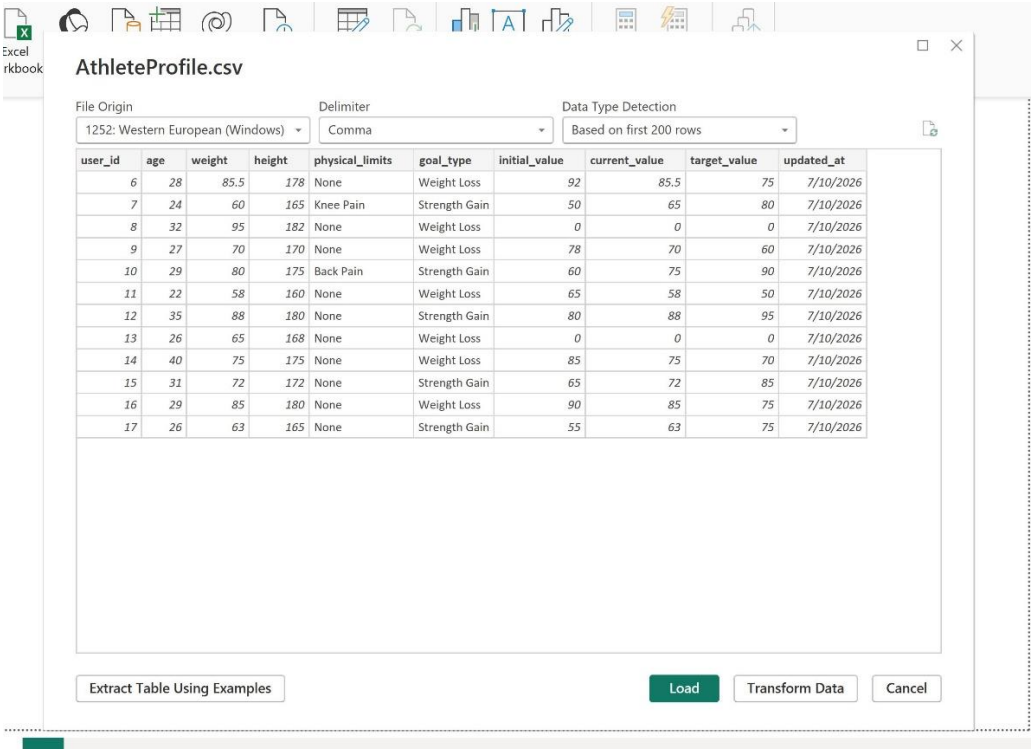
Feedbacks:

id	plan_id	content	send_date	is_read
1	2	Your progress is excellent! Increase the weight by 2.5kg.	2026-07-08	1
2	2	Don't forget the stretching exercises.	2026-07-05	0
3	4	Your squat depth has improved a lot. Keep it up.	2026-07-09	1
4	4	Please log your workouts more regularly.	2026-07-01	1
5	7	Amazing progress on pull-ups! We will add weight soon.	2026-07-07	1
6	7	Did you do your cardio today?	2026-07-04	0
7	9	Your weight loss goal is going great. Keep the protein high.	2026-07-09	1
8	9	Increase water intake to 3 liters per day.	2026-07-02	1
9	12	It seems you missed leg day. Are you in any pain?	2026-07-06	0
10	15	You are one of my best athletes! Continue this trend.	2026-07-08	1
11	17	We aim to hit a new deadlift PR next week.	2026-07-07	1
12	19	Your upper body strength is skyrocketing. Great!	2026-07-09	1

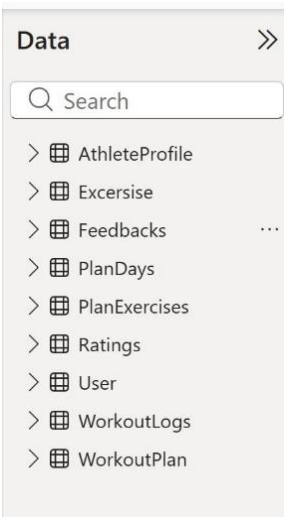
Ratings:

	A	B	C	D
1	id	plan_id	score	created_at
2	1	1	4	2026-06-10
3	2	2	5	2026-07-05
4	3	3	3	2026-06-25
5	4	4	4	2026-07-01
6	5	6	2	2026-06-20
7	6	7	5	2026-06-25
8	7	8	5	2026-06-30
9	8	9	4	2026-07-05
10	9	10	1	2026-07-02
11	10	11	2	2026-06-15

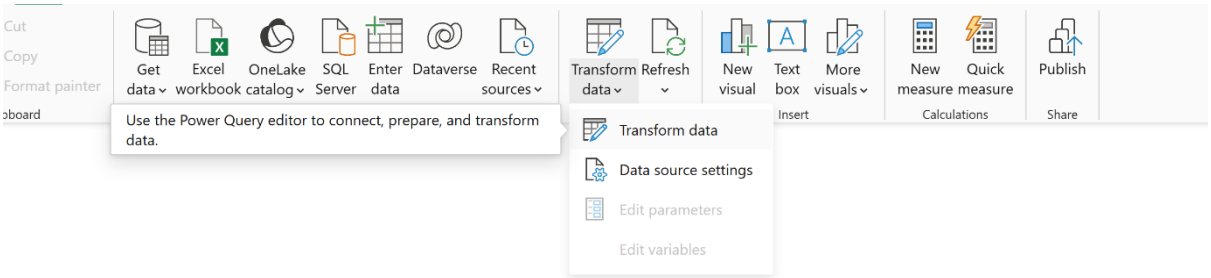
Then we import the data into Power BI and select the Load option so that the data is loaded.



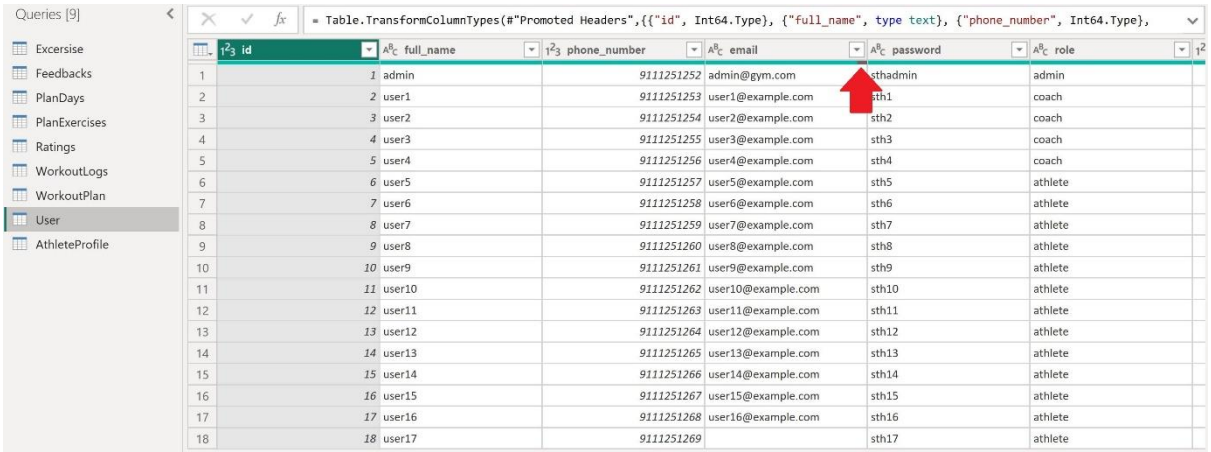
Data loaded:



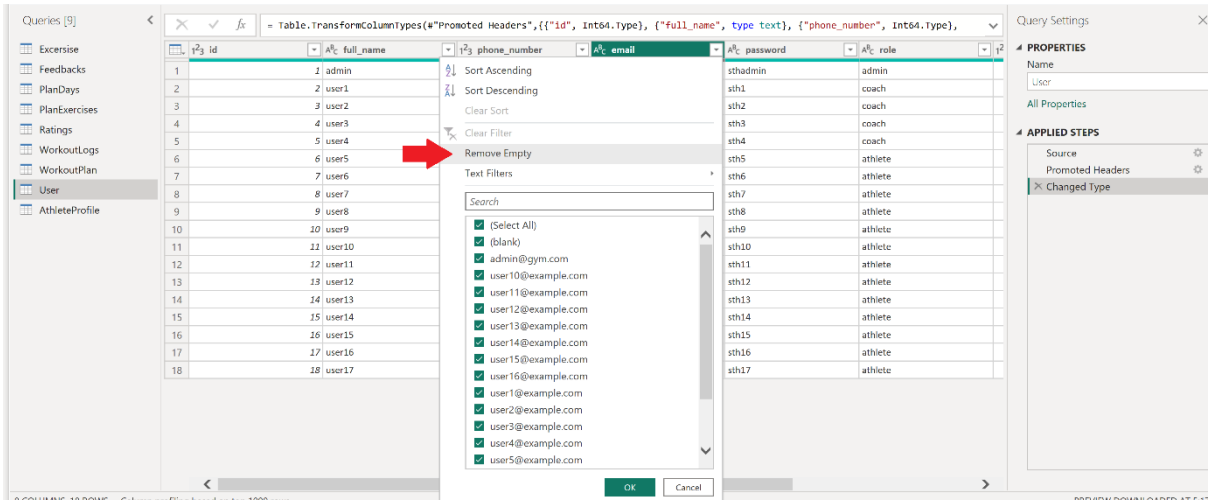
Next, in the Home tab, under the Queries group, click the Transform Data icon to open the Power Query Editor



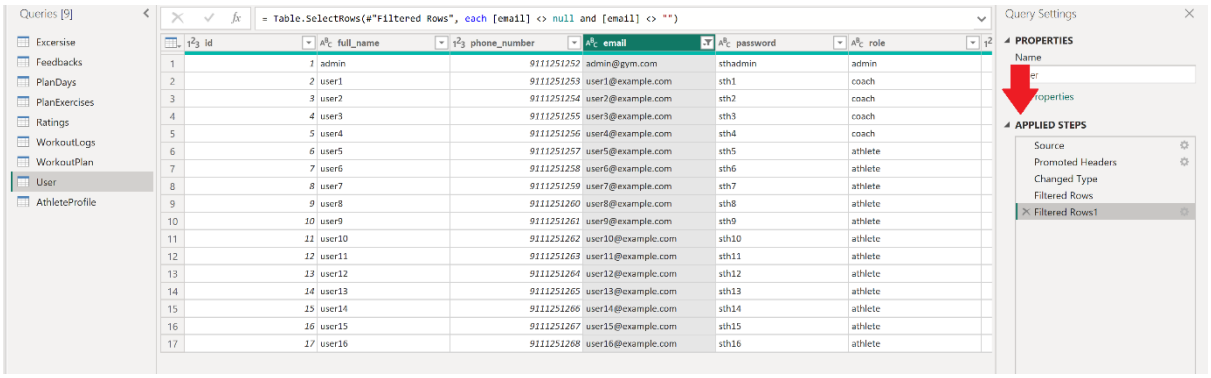
In the User table, below the column headers, a green line is visible. However, the line under the email column is not entirely green and has a small black section. This black section indicates that some values in these columns are empty (null). Therefore, it is recommended to remove or filter out the empty columns or rows.



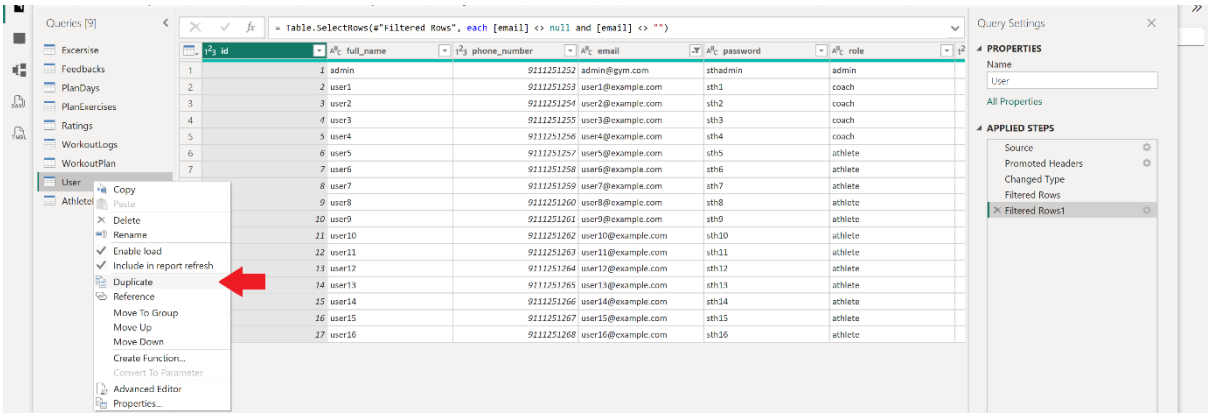
To remove the empty values, open the filter dropdown and select the Remove Empty option. This will delete all rows where the email column is empty from the table.



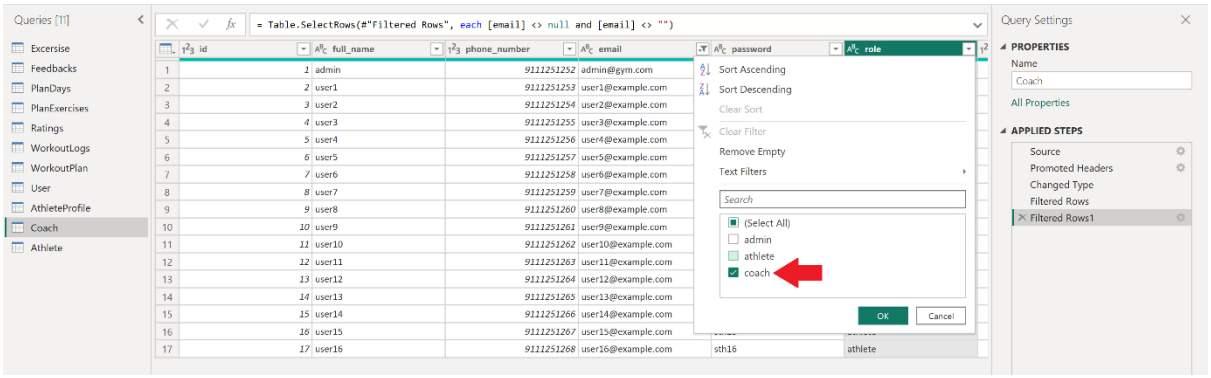
On the right side, in the Applied Steps section, there is a list of the performed steps. Up to the Changed Type step was done by Power BI itself, and the next steps were created by us by applying changes. In the Filtered Rows step, we defined that for the user file, the empty data in the email column should be removed.



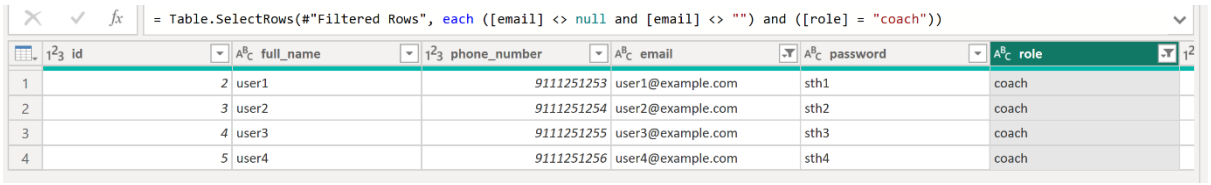
We split the user table into two tables, athlete and coach, so that no problem arises for the relationships (we no longer need the user table anymore):



Then we filter the data:

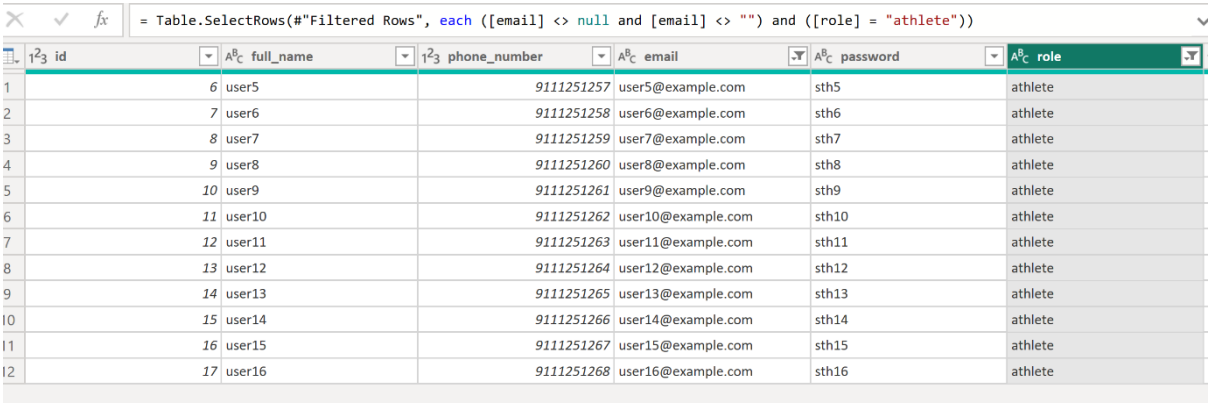


Coach and athlete tables:



= Table.SelectRows(#"Filtered Rows", each ([email] <> null and [email] <> "") and ([role] = "coach"))

	id	full_name	phone_number	email	password	role
1	2	user1	9111251253	user1@example.com	sth1	coach
2	3	user2	9111251254	user2@example.com	sth2	coach
3	4	user3	9111251255	user3@example.com	sth3	coach
4	5	user4	9111251256	user4@example.com	sth4	coach

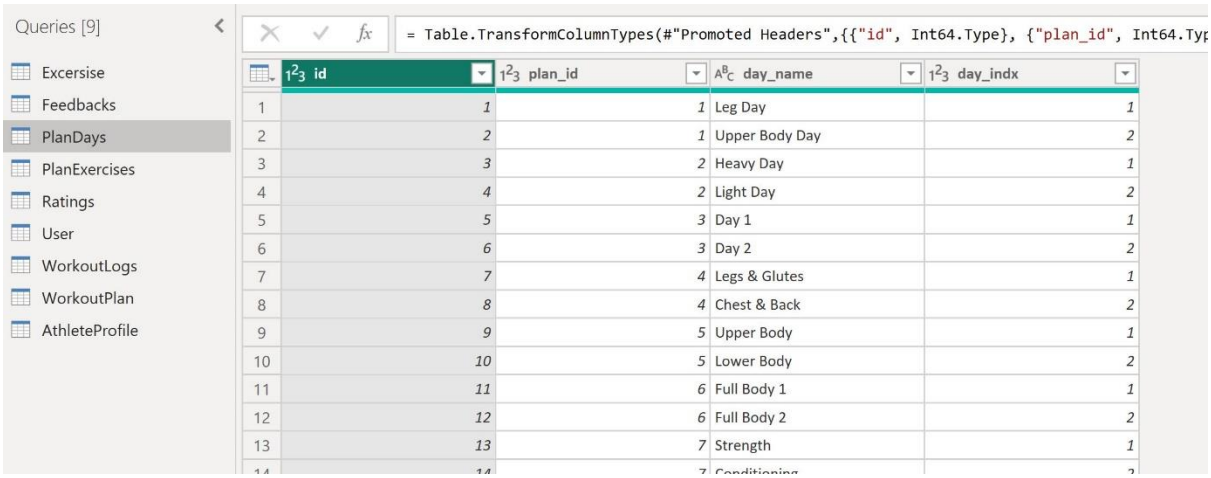


= Table.SelectRows(#"Filtered Rows", each ([email] <> null and [email] <> "") and ([role] = "athlete"))

	id	full_name	phone_number	email	password	role
1	6	user5	9111251257	user5@example.com	sth5	athlete
2	7	user6	9111251258	user6@example.com	sth6	athlete
3	8	user7	9111251259	user7@example.com	sth7	athlete
4	9	user8	9111251260	user8@example.com	sth8	athlete
5	10	user9	9111251261	user9@example.com	sth9	athlete
6	11	user10	9111251262	user10@example.com	sth10	athlete
7	12	user11	9111251263	user11@example.com	sth11	athlete
8	13	user12	9111251264	user12@example.com	sth12	athlete
9	14	user13	9111251265	user13@example.com	sth13	athlete
10	15	user14	9111251266	user14@example.com	sth14	athlete
11	16	user15	9111251267	user15@example.com	sth15	athlete
12	17	user16	9111251268	user16@example.com	sth16	athlete

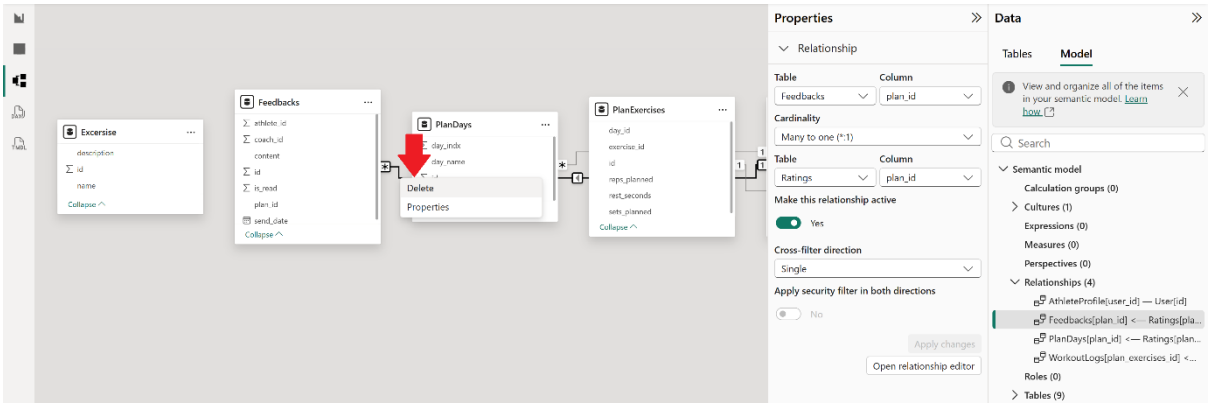
Now, if new data is added, the tables update automatically.

The rest of the tables are correct (a completely green line for all columns):



	id	plan_id	day_name	day_indx
1	1	1	Leg Day	1
2	2	1	Upper Body Day	2
3	3	2	Heavy Day	1
4	4	2	Light Day	2
5	5	3	Day 1	1
6	6	3	Day 2	2
7	7	4	Legs & Glutes	1
8	8	4	Chest & Back	2
9	9	5	Upper Body	1
10	10	5	Lower Body	2
11	11	6	Full Body 1	1
12	12	6	Full Body 2	2
13	13	7	Strength	1

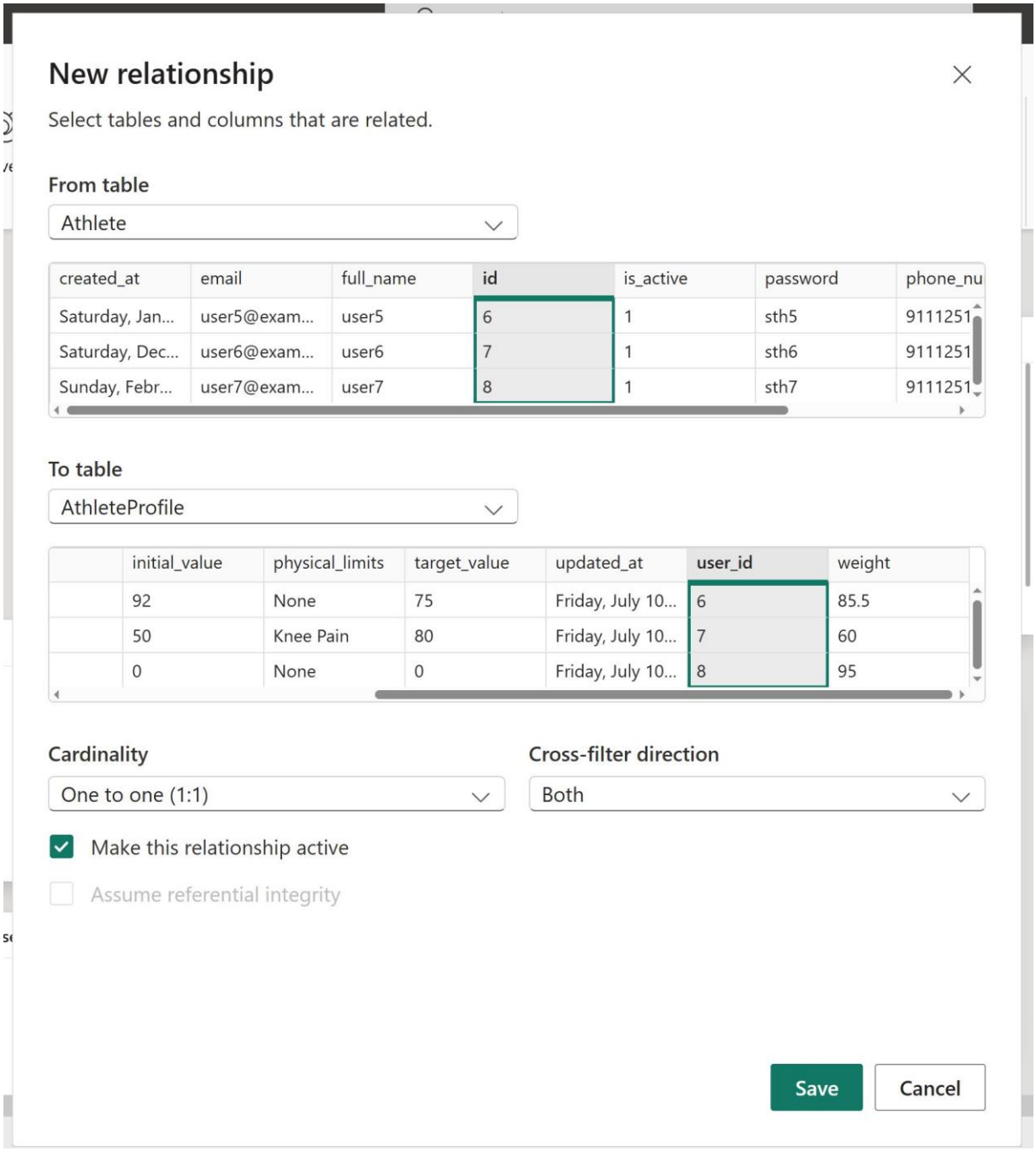
Next, we need to define the relationships between the tables. We navigate to the Model view and remove any auto-detected relationships, as they may not be accurate.



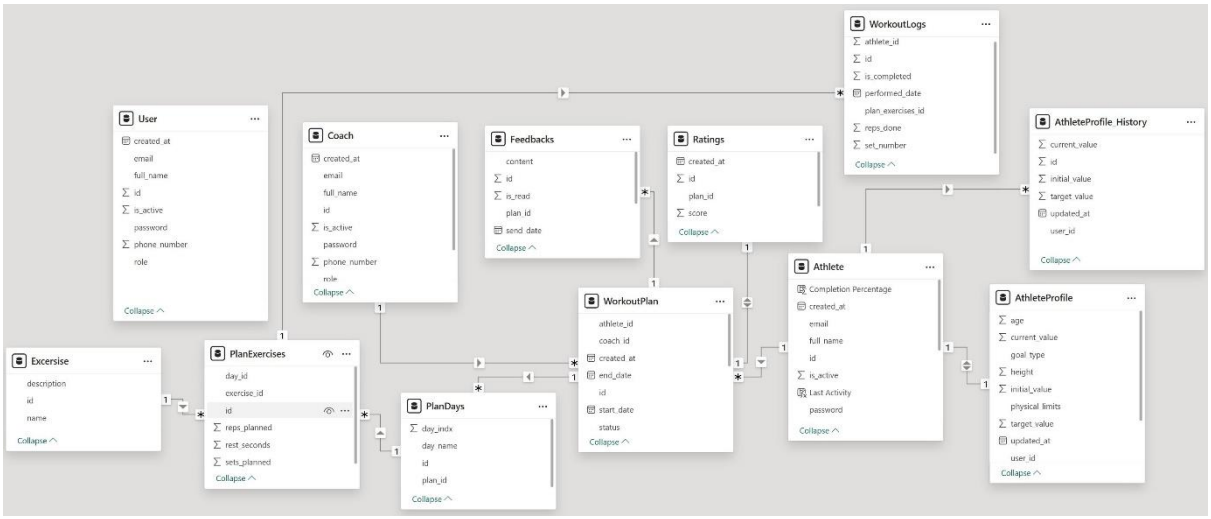
Relationships between tables:

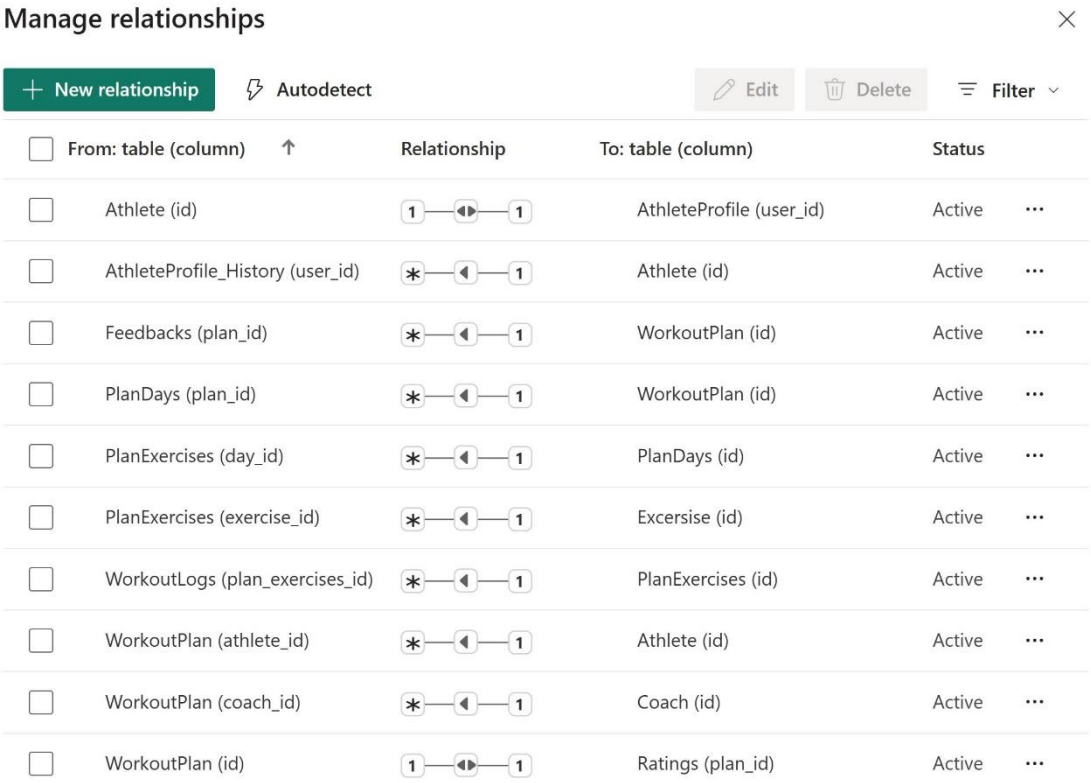
PrimaryKey	ForeignKey	Relationship Type	detail
Athlete.id	AthleteProfile.user_id	One to one	Each athlete has only one profile
Coach.id	WorkoutPlan.coach_id	One to many	Each program is created by one coach
Athlete.id	WorkoutPlan.athlete_id	One to many	Each program is written for one athlete
WorkoutPlan.id	PlanDays.plan_id	One to many	Each program consists of several training sessions
PlanDays.id	PlanExercises.day_id	One to many	Each session consists of several exercises
Excersise.id	PlanExercises.exercise_id	One to many	Each exercise is connected to the main exercise list
PlanExercises.id	WorkoutLogs.plan_exercises_id	One to many	Each workout log belongs to a specific exercise
WorkoutPlan.id	Feedbacks.plan_id	One to many	Each feedback belongs to a program
WorkoutPlan.id	Ratings.plan_id	One to many	Each rating belongs to a program
Athlete.id	AthleteProfile_History.uers_id	One to many	Each athlete has several history records

To add relationships, we draw the relationship from the primary key side to the foreign key side. To do this, we drag and drop the corresponding column onto the next column, and then specify the relationship type:



Relationships:





Then for the reports, we need to use DAX, which is a powerful formula language in Power BI that is used for performing dynamic and complex calculations. Unlike regular columns, DAX Measures are calculated at the moment the report is displayed, taking into account the selected filters, and are not stored in the database. This language, by providing various functions such as SUM, AVERAGE, CALCULATE, and FILTER, enables advanced analysis, creating interactive reports, and transforming raw data into valuable information.

Reports:

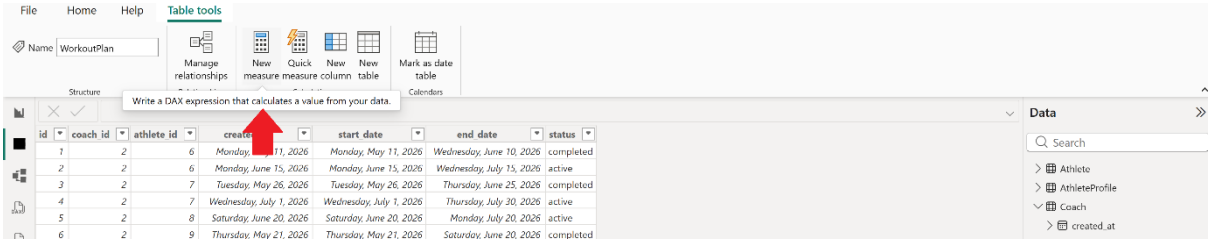
1. Coach Performance Dashboard

Description: This report evaluates coach performance based on their interactions with athletes over a specified time period.

Fields:

1) Coach Name: Full name of the coach.
2) Active Athletes: Number of athletes currently enrolled in an active program with this coach.
3) Total Programs Sent: Total number of programs created by the coach (both active and completed).
4) Total Feedbacks Sent: Total number of feedback messages sent by the coach.
5) Average Rating: Average rating (from 1 to 5) given by athletes to this coach.
6) Average Response Time (Program Creation to Feedback Submission): Average number of days between the program creation date and the feedback submission date (lower is better).
7) Response Rate: Percentage of programs that have received at least one feedback message (higher indicates better engagement).
8) Athlete Retention Rate: Percentage of athletes who are still active compared to the total number of athletes who have ever worked with this coach.

To do this, in Table View, for example, on the WorkoutPlan table, we select New Measure (we add each measure to the table it is most related to):



Then we calculate the required formulas using the following functions:

The COUNTROWS function counts the number of rows in a table.

```
COUNTROWS ( [<Table>] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Table	Optional	The table containing the rows to be counted. If it is not specified, it uses the table containing the measure definition.

The DISTINCTCOUNT function counts the number of unique values present in a specified column of a table.

```
DISTINCTCOUNT ( <ColumnName> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
ColumnName		The column for which the distinct values are counted.

The AVERAGE function calculates the average of the values in a specified column of a table.

```
AVERAGE ( <ColumnName> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
ColumnName		The column that contains the numbers for which you want the average.

The CALCULATE function executes an expression based on specified filters.

```
CALCULATE ( <Expression> [, <Filter> [, <Filter> [, … ] ] ] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Expression BY EXPRESSION ⓘ		The expression to be evaluated.
Filter	Optional Repeatable	A boolean (True/False) expression or a table expression that defines a filter.

The MAX function returns the largest value from a column.

```
MAX ( <ColumnNameOrScalar1> [, <Scalar2>] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
ColumnNameOrScalar1		The column in which you want to find the largest value, or the first scalar expression to compare.
Scalar2	Optional	The second value to compare.

The SWITCH function.

```
SWITCH ( <Expression>, <Value>, <Result> [, <Value>, <Result> [, … ] ] [, <Else>] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Expression		The expression to be evaluated.
Value	Repeatable	If expression has this value the corresponding result will be returned.
Result	Repeatable	The result to be returned if Expression has corresponding value.
Else	Optional	If there are no matching values the Else value is returned.

The DATEDIFF function calculates the number of specified time units between two input dates.

```
DATEDIFF ( <Date1>, <Date2>, <Interval> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Date1		A date in datetime format that represents the start date.
Date2		A date in datetime format that represents the end date.
Interval		The unit that will be used to calculate, between the two dates. It can be SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR.

The AVERAGEX function traverses row by row in a table and executes a formula for each row, and then returns the average of all those results.

```
AVERAGEX ( <Table>, <Expression> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Table ITERATOR		The table containing the rows for which the expression will be evaluated.
Expression ROW CONTEXT BY EXPRESSION		The expression to be evaluated for each row of the table.

The RELATED function receives and returns a related value from another table.

```
RELATED ( <ColumnName> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
ColumnName		The column that contains the desired value.

The DEVIDE function.

```
DIVIDE ( <Numerator>, <Denominator> [, <AlternateResult>] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Numerator		Numerator.
Denominator		Denominator.
AlternateResult	Optional	Optional. The alternate result to return when dividing by zero.

The IF function.

```
IF ( <LogicalTest>, <ResultIfTrue> [, <ResultIfFalse>] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
LogicalTest		Any value or expression that can be evaluated to TRUE or FALSE.
ResultIfTrue		The value that is returned if the logical test is TRUE.
ResultIfFalse	Optional	The value that is returned if the logical test is FALSE. If omitted, a BLANK is returned.

The ALL function: Returns all rows of a table or all values of a column, and ignores any filters that may have been applied.

```
ALL ( [<TableNameOrColumnName>] [, <ColumnName> [, <ColumnName>
[, … ] ] ] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
TableNameOrColumnName	Optional	The name of an existing table or column.
ColumnName	Optional Repeatable	A column in the same base table. The column can be specified in optional parameters only when a column is used in the first argument, too.

The SUM function adds up all the values in a column of a table.

```
SUM ( <ColumnName> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
ColumnName		The column that contains the numbers to sum.

The SUMX function: First, for each row of the table, it performs a new calculation, and then it adds the results together.

```
SUMX ( <Table>, <Expression> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Table ITERATOR ⓘ		The table containing the rows for which the expression will be evaluated.
Expression ROW CONTEXT ⓘ BY EXPRESSION ⓘ		The expression to be evaluated for each row of the table.

The ALLSELECTED function: returns all rows of a table or all values of a column, but ignores the filters that are applied within the query (or current context), while retaining the filters that come from outside (such as Slicers)

```
ALLSELECTED ( [<TableNameOrColumnName>] [, <ColumnName> [, <ColumnName> [, … ] ] ] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
TableNameOrColumnName	Optional	Remove all filters on the specified table or column applied within the query.
ColumnName	Optional Repeatable	A column in the same base table.

The VALUES function returns a table. The behavior of this function depends on the type of its input:

- If the input is a Column, it returns a single-column table of the unique and non-duplicate values of that column.
- If the input is a Table, it returns all rows of that table with the same columns (including duplicate values), plus a blank row.

```
VALUES ( <TableNameOrColumnName> )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
TableNameOrColumnName		A column name or a table name.

The CALCULATETABLE function: evaluates and executes a table expression in a Context that has been modified by the specified filters.

```
CALCULATETABLE ( <Table> [, <Filter> [, <Filter> [, … ] ] ] )
```

PARAMETER	ATTRIBUTES	DESCRIPTION
Table BY EXPRESSION ⓘ		The table expression to be evaluated.
Filter	Optional Repeatable	A boolean (True/False) expression or a table expression that defines a filter.

Formula:

Active Athletes per Coach:

Active Athletes per Coach =
CALCULATE(DISTINCTCOUNT(WorkoutPlan[athlete_id]),WorkoutPlan[status]= "active")

Number of Programs Sent:

Programs sent = COUNTROWS(WorkoutPlan)

Number of Feedbacks Sent:

Feedbacks sent = COUNTROWS(Feedbacks)

Avg Rating:

Avg Rating = AVERAGE(Ratings[score])

Response Percentage:

Response Percentage =
VAR PlansWithFeedback = COUNTROWS(FILTER(WorkoutPlan,
COUNTROWS(RELATEDTABLE(Feedbacks)) > 0))
VAR TotalPlans = COUNTROWS(WorkoutPlan)
RETURN DIVIDE(PlansWithFeedback, TotalPlans, 0)

The ratio of the number of programs that have received at least one feedback to the total number of programs sent by the coach.

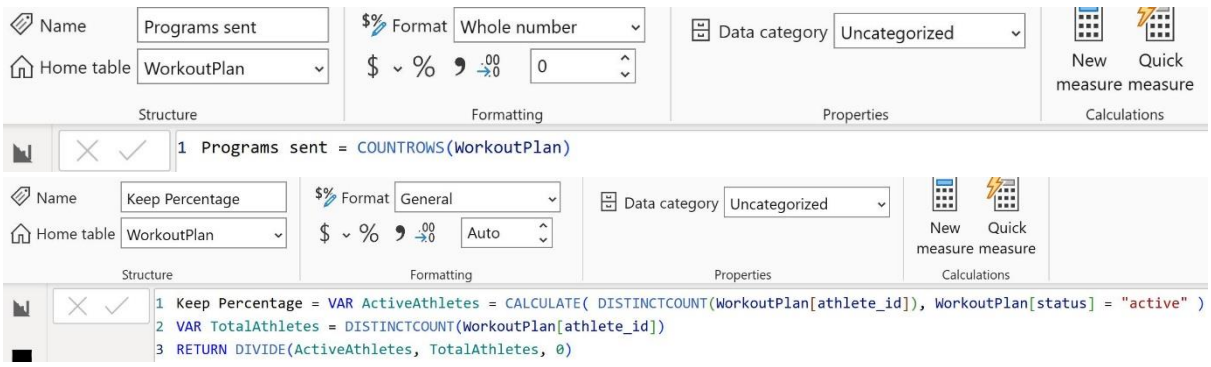
Avg Response Time (Days) = AVERAGEX(Feedbacks, DATEDIFF(
RELATED(WorkoutPlan[created_at]), Feedbacks[send_date], DAY))

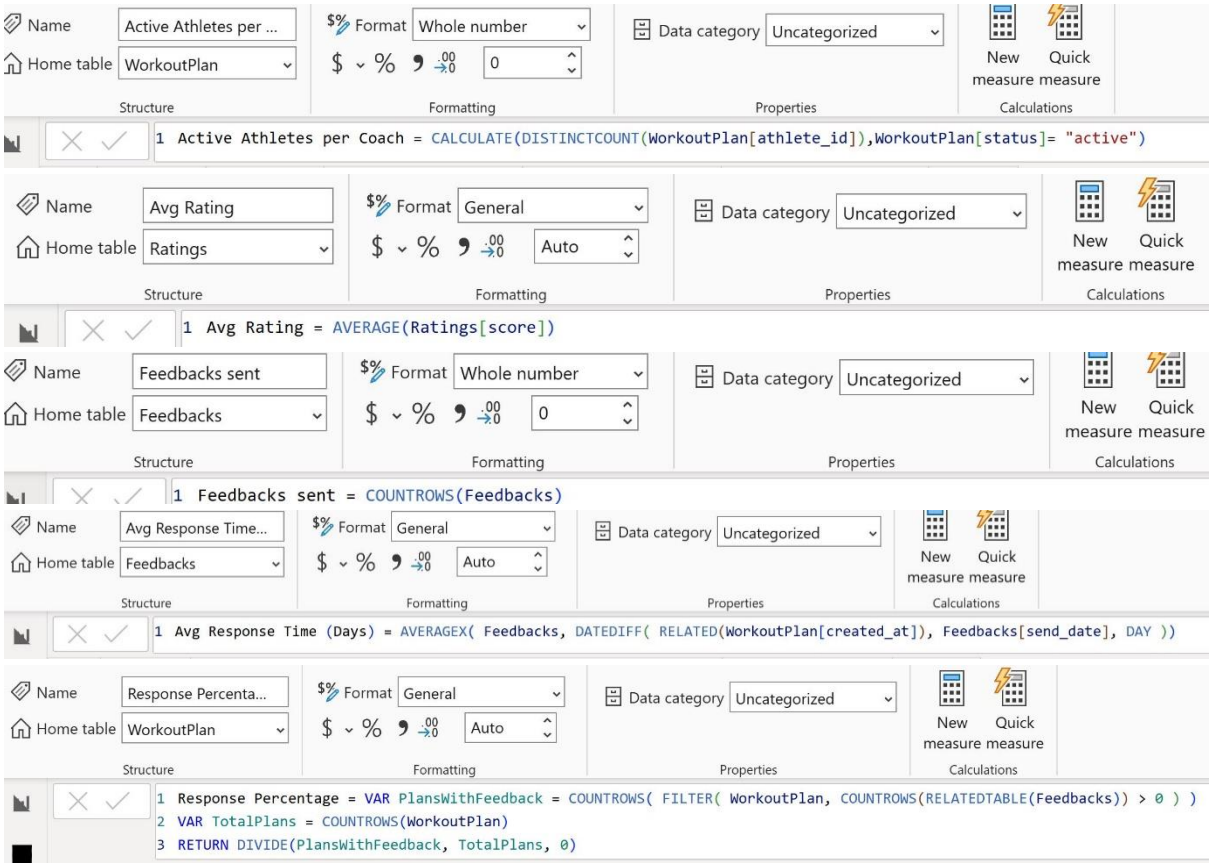
Calculates the average number of days that pass from the time a program is created by the coach until the time the coach sends feedback (message) for that program. It is executed row by row on the Feedbacks table, and then the average is calculated.

Athlete Retention Rate:

Keep Percentage =
VAR ActiveAthletes = CALCULATE(DISTINCTCOUNT(WorkoutPlan[athlete_id]),
WorkoutPlan[status] = "active")
VAR TotalAthletes = DISTINCTCOUNT(WorkoutPlan[athlete_id])
RETURN DIVIDE(ActiveAthletes, TotalAthletes, 0)

Adding the measures:





2. Athlete Consistency Dashboard

Description: A report on active athletes based on their program completion percentage.

Fields:

1) Athlete Name
2) Program Completion Percentage
3) Last Activity Date: The most recent date the athlete logged an exercise in the system.

Formula:

Completion Percentage:

This metric is calculated separately for each athlete and requires direct access to the athlete's ID in the same row. Therefore, instead of a measure, we use Add Column on the Athlete table so that the calculation is performed independently of relationships and correctly for all athletes.

also, in metrics that reference these types of columns, the AVERAGE function is used to retrieve each athlete's value, as it returns the unique value corresponding to that athlete in the current row.

Completion Percentage =

VAR AthleteID = Athlete[id]

VAR AthleteLogs = FILTER(WorkoutLogs,WorkoutLogs[athlete_id] = AthleteID)

VAR TotalSets = COUNTROWS(AthleteLogs)

VAR CompletedSets = COUNTROWS(FILTER(AthleteLogs,WorkoutLogs[is_completed] = 1))

RETURN IF(ISBLANK(TotalSets) || TotalSets = 0, 0, DIVIDE(CompletedSets, TotalSets, 0) * 100)

Divides the number of completed sets by the number of total scheduled sets.

Last Activity Date:

This metric is calculated separately for each athlete and requires direct access to the athlete's ID in the same row. Therefore, instead of a measure, we use Add Column on the Athlete table so that the calculation is performed independently of relationships and correctly for all athletes.

also, in metrics that reference these types of columns, the AVERAGE function is used to retrieve each athlete's value, as it returns the unique value corresponding to that athlete in the current row.

Last Activity =

VAR AthleteID = Athlete[id]

RETURN CALCULATE(MAX(WorkoutLogs[performed_date]),FILTER(WorkoutLogs,WorkoutLogs[athlete_id] = AthleteID))

Athlete Category:

Athlete Category = SWITCH(TRUE(), [Completion Percentage] >= 80, "High", [Completion Percentage] >= 50, "Medium", "Low")

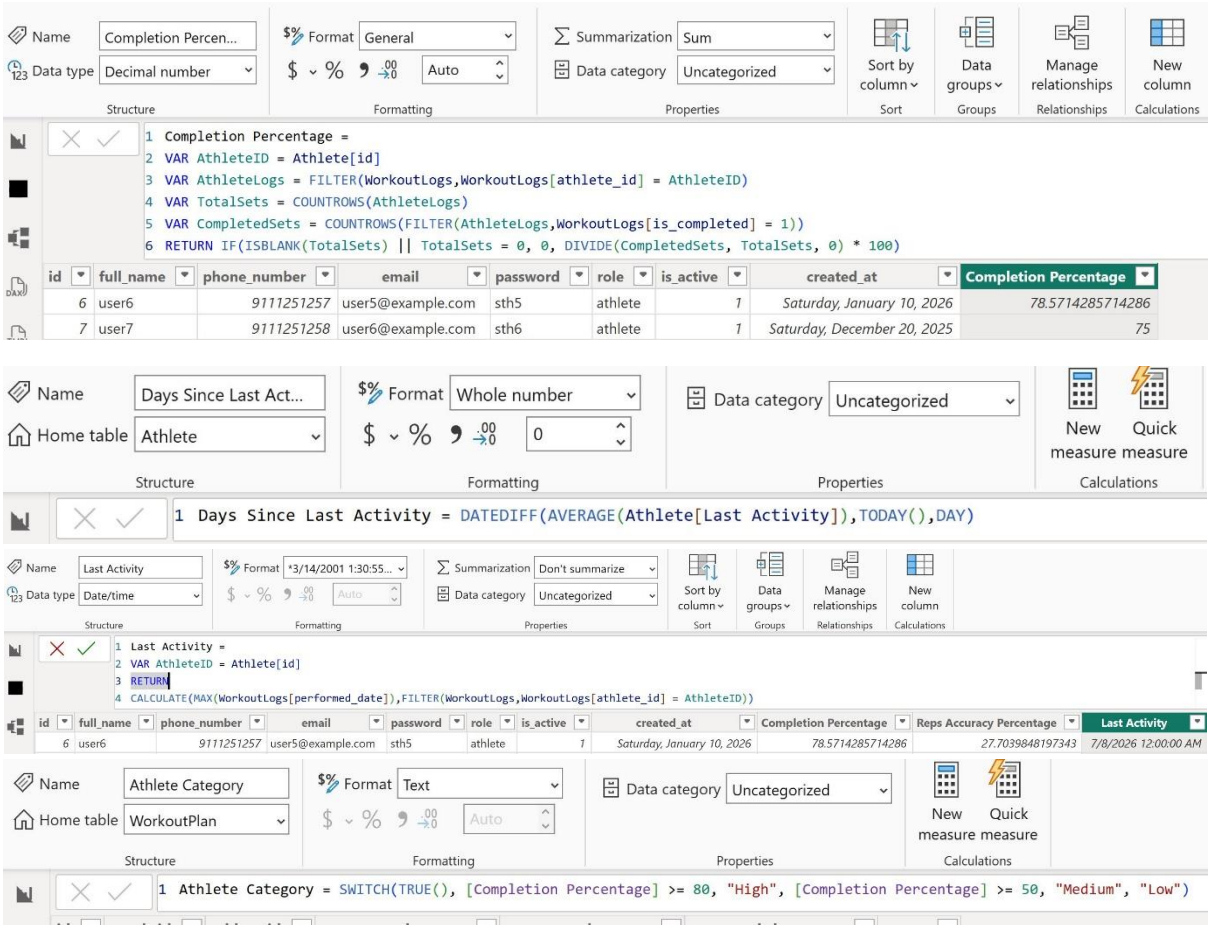
Athletes are divided into three categories:

- High: Completion percentage ≥ 80%
- Medium: Completion percentage between 50% and 80%
- Low: Completion percentage < 50%

Days Since Last Activity:

Days Since Last Activity = DATEDIFF(AVERAGE(Athlete[Last Activity]),TODAY(),DAY)

Adding the measures:



3. Club Status

Description: summary of athlete and coach status within a specified time period.

Fields:

1) Active Coaches
2) Active Athletes
3) Athlete-to-Coach Ratio
4) Total Programs Sent
5) Total Feedbacks Sent
6) Overall Average Coach Rating

Formula:

Number of Active Coaches:

Active Coaches = CALCULATE(COUNTROWS(Coach), Coach[is_active] = 1)

Number of Active Athletes:

Active Athletes = CALCULATE(COUNTROWS(Athlete), Athlete[is_active] = 1)

Athlete to Coach Ratio:

Athlete to Coach Ratio = DIVIDE([Active Athletes], [Active Coaches], 0)

Overall Programs Sent (we already have it):

Programs Sent = COUNTROWS(WorkoutPlan)

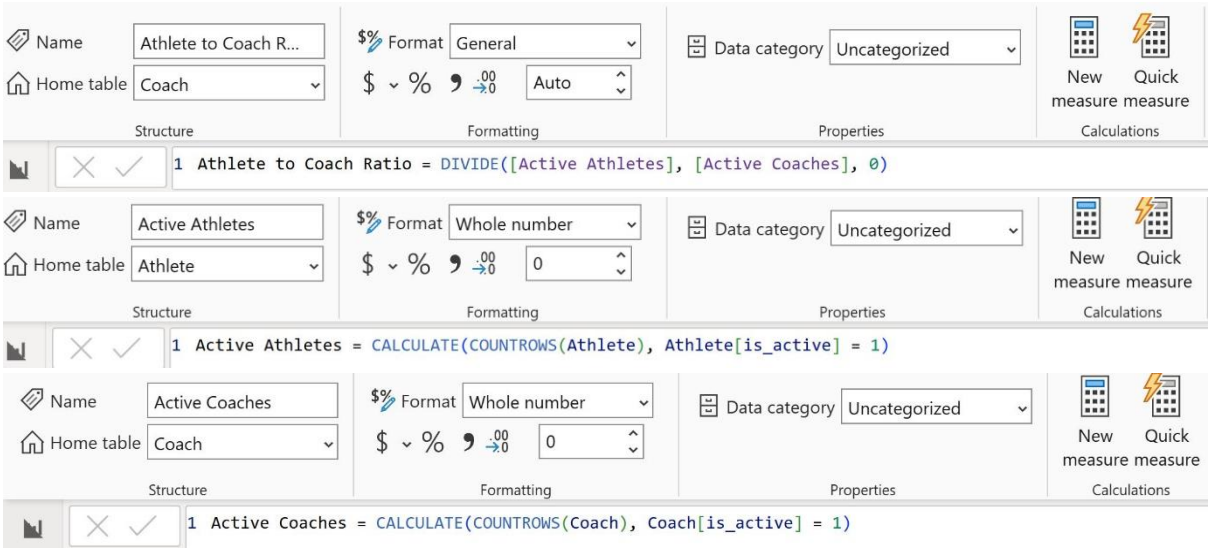
Overall Feedbacks Sent (we already have it):

Feedbacks Sent = COUNTROWS(Feedbacks)

Overall average rating of all coaches (we already have it):

Avg Rating = AVERAGE(Ratings[score])

Adding the measures:



4. At-Risk Athlete Identification

Description: Identifies athletes who may be declining in performance or engagement based on their program completion, accuracy, activity, and feedback metrics.

Fields:

1) Athlete Name
2) Coach Name
3) Collaboration Start Date
4) Average Program Completion Percentage
5) Average Programs Sent (per Month)
6) Days Since Last Activity

Formula:

Completion Percentage: Indicates what percentage of the sets the athlete started they actually completed (did not abandon the set). (Already created earlier.)

Completion Percentage =

VAR AthleteID = Athlete[id]

VAR AthleteLogs = FILTER(WorkoutLogs,WorkoutLogs[athlete_id] = AthleteID)

VAR TotalSets = COUNTROWS(AthleteLogs)

VAR CompletedSets = COUNTROWS(FILTER(AthleteLogs,WorkoutLogs[is_completed] = 1))

RETURN IF(ISBLANK(TotalSets) || TotalSets = 0, 0, DIVIDE(CompletedSets, TotalSets, 0) * 100)

Reps Accuracy Percentage: Indicates what percentage of the reps prescribed by the coach the athlete actually performed.

This metric is calculated separately for each athlete and requires direct access to the athlete's ID in the same row. Therefore, instead of a measure, we use Add Column on the Athlete table so that the calculation is performed independently of relationships and correctly for all athletes.

Reps Accuracy Percentage =

VAR AthleteID = Athlete[id]

VAR TotalRepsDone =SUMX(FILTER(WorkoutLogs, WorkoutLogs[athlete_id] = AthleteID),WorkoutLogs[reps_done])

VAR TotalRepsPlanned = CALCULATE(SUMX(PlanExercises,PlanExercises[sets_planned] * PlanExercises[reps_planned]),PlanDays,WorkoutPlan,WorkoutPlan[athlete_id] = AthleteID)

RETURN IF(ISBLANK(TotalRepsPlanned) || TotalRepsPlanned = 0,0,DIVIDE(TotalRepsDone, TotalRepsPlanned, 0) * 100)

Overall Performance: The average of the two metrics (Completion Percentage and Reps Accuracy Percentage), indicating how successful the athlete has been overall.

Overall Performance =

VAR CompletionWeight = 0.5

VAR RepsWeight = 0.5

VAR CompletionScore = AVERAGE(Athlete[Completion Percentage])

VAR RepsScore = AVERAGE(Athlete[Reps Accuracy Percentage])

RETURN (CompletionWeight * CompletionScore) + (RepsWeight * RepsScore)

Performance Level: Categorizes athletes into four performance levels: Excellent, Average, Needs Attention, Critical.

Note: Since we want to use Performance Level in a donut chart, it must be a Calculated Column, not a Measure.

Performance Level =

```
VAR Overall = [Overall Performance]
RETURN SWITCH( TRUE(),
    Overall >= 80, "Excellent",
    Overall >= 60, "Average ",
    Overall >= 40, "Needs Attention ",
    "Critical ")
```

Performance Diagnosis: Identifies the reason for an athlete's decline or success. For example, it can diagnose that an athlete is putting in low effort (they complete the sets but perform fewer reps than prescribed).

Performance Diagnosis =

```
VAR Comp = AVERAGE(Athlete[Completion Percentage])
VAR Reps = AVERAGE(Athlete[Reps Accuracy Percentage])
RETURN SWITCH(TRUE(),
    Comp >= 80 && Reps >= 80, "excellent (complete and accurate)",
    Comp >= 80 && Reps < 80, "low effort (sets complete, reps low)",
    Comp < 80 && Reps >= 80, "overloaded (reps accurate but sets incomplete)",
    Comp < 80 && Reps < 80, "critical (both sets and reps low)")
```

Average Programs Sent:

```
Months Active = DATEDIFF(MIN(WorkoutPlan[start_date]),TODAY(),MONTH)
Avg Programs Sent (per Month) = DIVIDE( [Programs Sent], [Months Active],0)
```

Months Active calculates the number of months that have passed from the first time an athlete received a training program up to today. This represents the total number of full months the athlete has been collaborating with the coach.

If the Avg Programs Sent number is low, it indicates that the coach has not written enough programs for the athlete, and this could be one of the reasons for the athlete's lack of motivation.

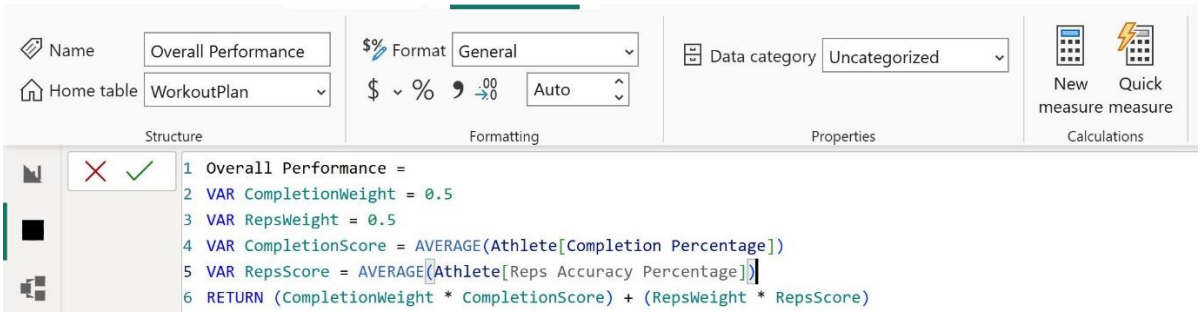
Days Since Last Activity (we already have it):

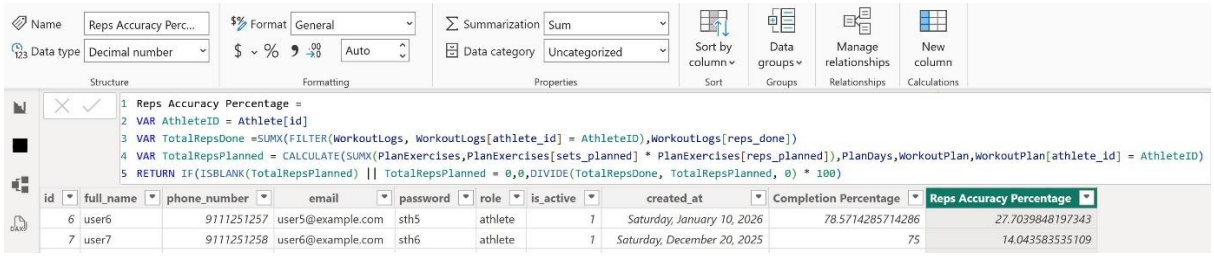
```
Days Since Last Activity = DATEDIFF([Last Activity], TODAY(), DAY)
```

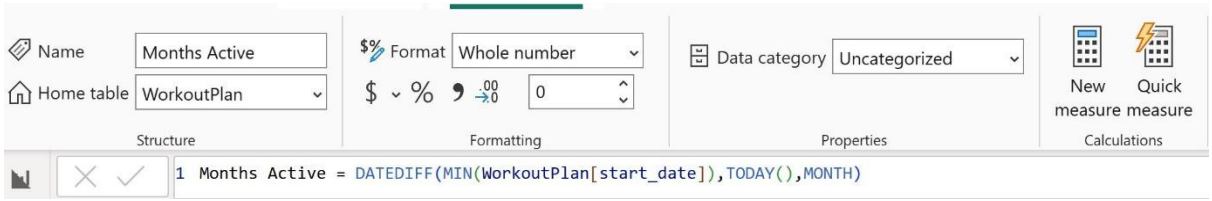
Number of Unread Messages:

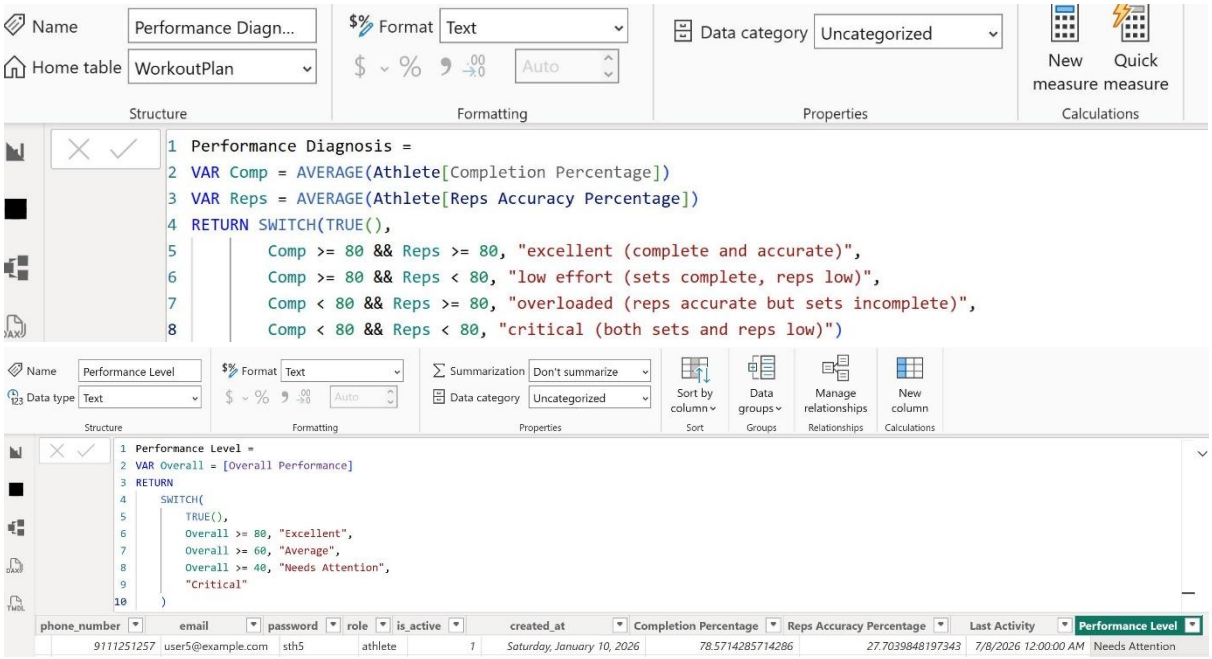
```
Unread Messages = CALCULATE(COUNTROWS(Feedbacks), Feedbacks[is_read] = 0)
```

Adding the measures:









5. Athlete Goal Achievement

Description: A report on athlete progress toward their defined goals.

Fields:

1) Athlete Name
2) Coach Name
3) Goal Type
4) Initial Value
5) Current Value
6) Progress Percentage
7) Estimated Time to Reach Goal

Formula:

Goal Progress Percentage

Goal Progress Percentage =

VAR Init = AVERAGE(AthleteProfile[initial_value])

VAR Curr = AVERAGE(AthleteProfile[current_value])

VAR Targ = AVERAGE(AthleteProfile[target_value])

RETURN DIVIDE(Curr - Init, Targ - Init, 0) * 100

A value of 100 means the athlete has reached their goal.

Estimated Time to Goal (Days):

Estimated Time to Goal (Days) =

VAR CurrentAthlete = SELECTEDVALUE(Athlete[id])

VAR CurrentProgress = [Goal Progress Percentage]

VAR FirstLogDate =

CALCULATE(MIN(WorkoutLogs[performed_date]),WorkoutLogs[athlete_id] = CurrentAthlete,ALL(WorkoutLogs))

VAR DaysElapsed = DATEDIFF(FirstLogDate, TODAY(), DAY)

VAR Remain = 100 - CurrentProgress

RETURN IF(CurrentProgress > 0 &&NOT ISBLANK(FirstLogDate),DIVIDE(DaysElapsed * Remain, CurrentProgress, 0),BLANK())

DaysElapsed: The time from the first logged workout (indicating the actual start of the athlete's activity) to today.

Remain: The remaining percentage needed to reach the goal.

This calculates the approximate number of days until the athlete reaches their goal (assuming they have at least one logged workout), based on their current rate of progress. In other words, if the athlete continues at this pace, how many more days until they reach their goal.

Goal Progress (History): Calculates the percentage of progress an athlete has made toward their goal at any given point in time. It uses the initial value, current value, and target value from the AthleteProfile_History table and measures how far the athlete has traveled from start to finish.

Goal Progress (History) =

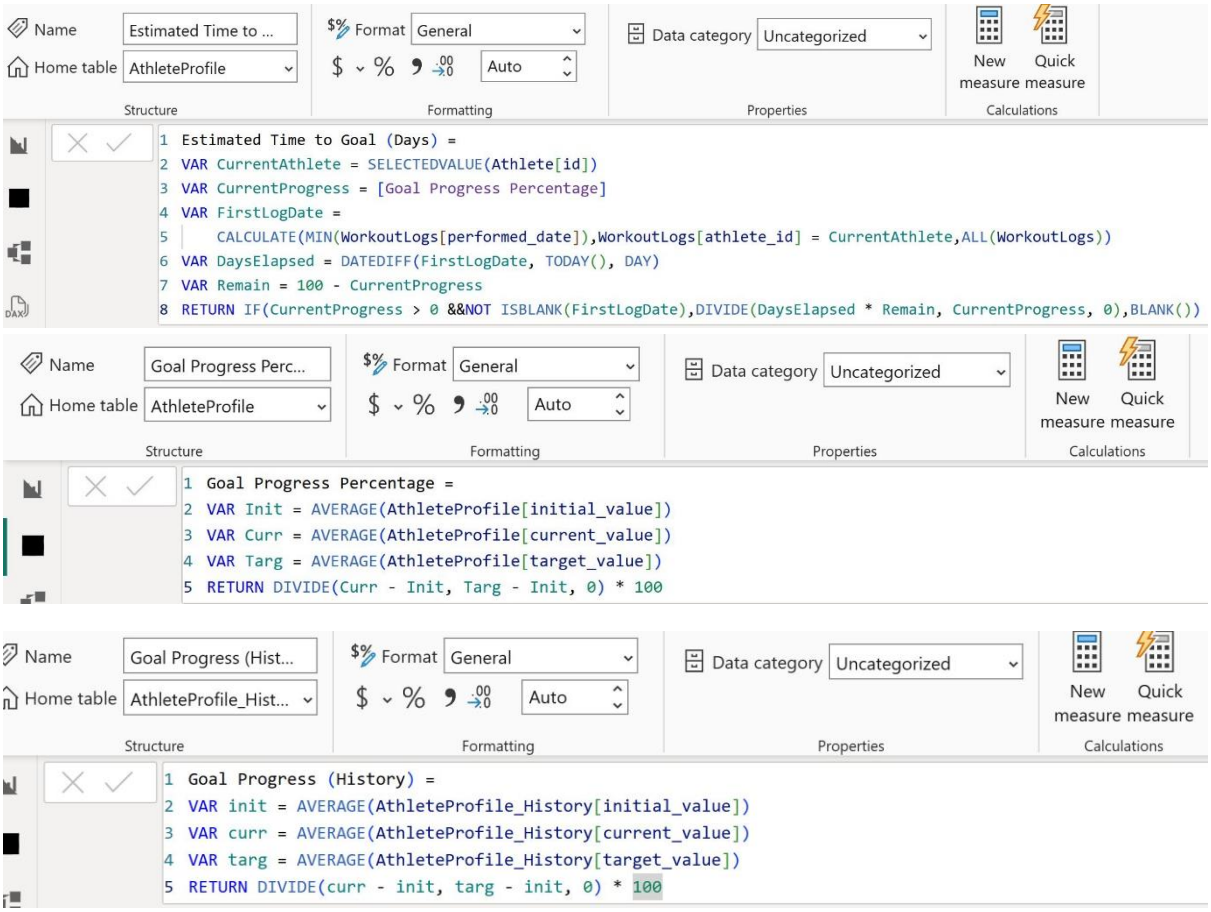
VAR init = AVERAGE(AthleteProfile_History[initial_value])

VAR curr = AVERAGE(AthleteProfile_History[current_value])

VAR targ = AVERAGE(AthleteProfile_History[target_value])

RETURN DIVIDE(curr - init, targ - init, 0) * 100

Adding the measures:



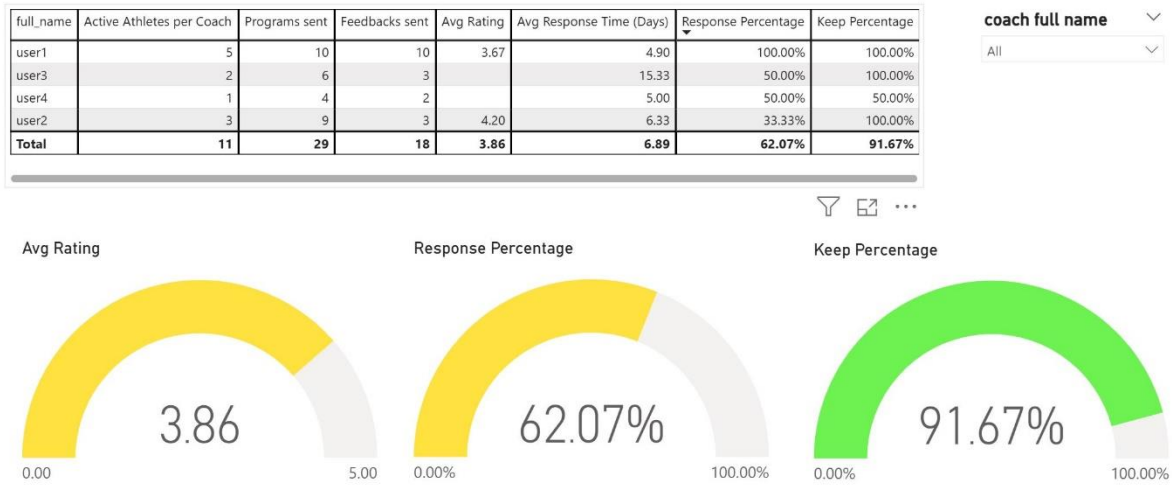
To display the chart, we go to the Report View section and place a chart on the page, then we fill the X-axis and Y-axis

1. Coach Performance Dashboard

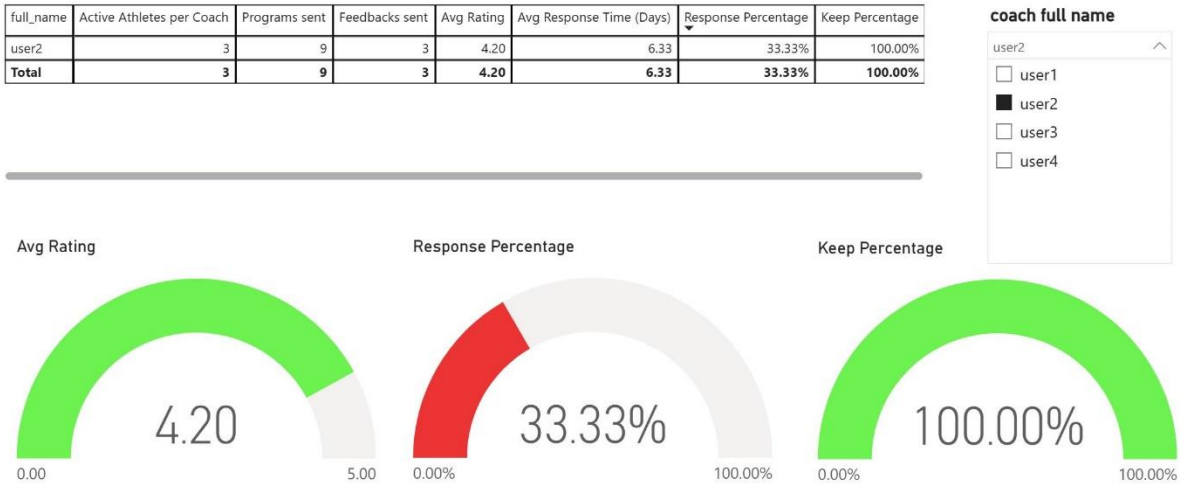
For the first report, we used a Slicer, three Gauges, and a Table.

The table clearly and numerically displays the following fields: Active Athletes, Programs Sent, Feedbacks Sent, Average Rating, Average Response Time, Response Percentage, and Athlete Retention Rate.

The gauges provide a quick, visual snapshot of coach performance without the need to read through the entire table. This allows management to understand at a glance whether a coach is performing well or not.



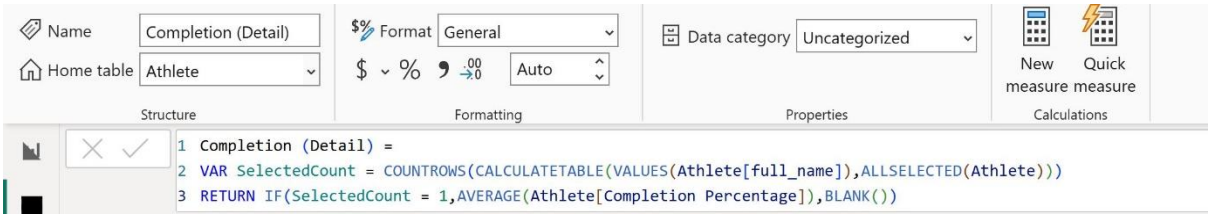
Selecting a specific coach with the slicer:



2. Athlete Consistency Dashboard

In the initial state, the table displays general information, and when a specific athlete is selected from the slicer, the table displays more detailed information. For this, we need three new measures to display this information:

For Completion:

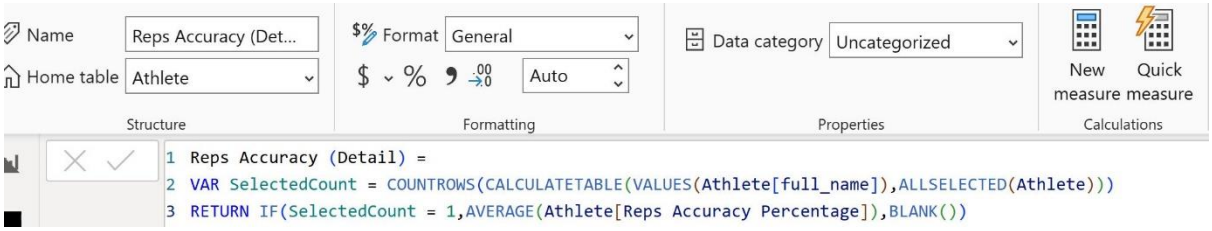


VALUES(Athlete[full_name]) returns the list of athlete names.

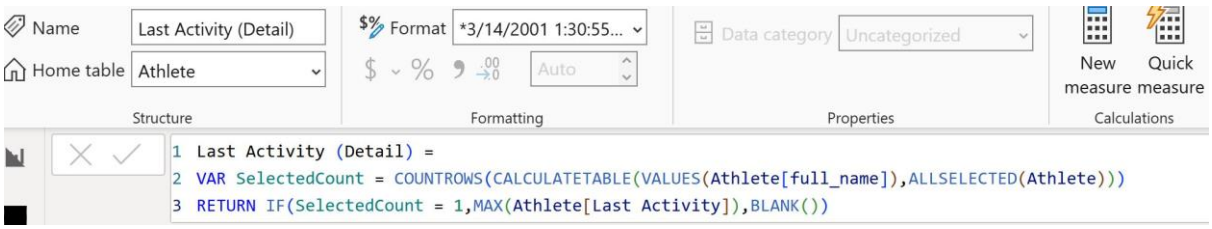
ALLSELECTED(Athlete) only considers external filters (slicers).

CALCULATETABLE returns the list of names in the slicer; if it contains only one value (single selection), the next section is executed.

for Reps accuracy:

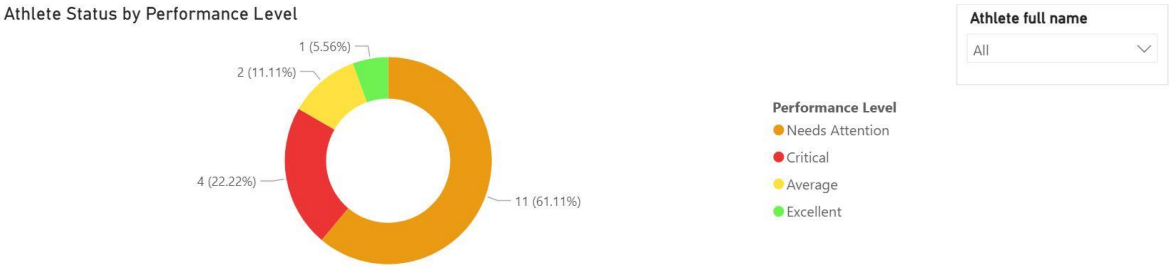


for Last activity:



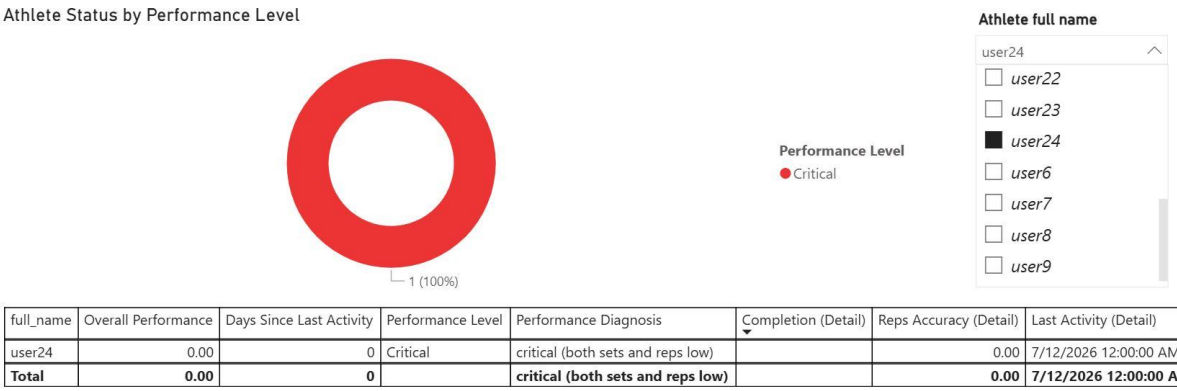
So in the slicer, when no athlete is selected, the detail columns are empty, but when a specific athlete is selected, their full details are displayed.

When all are selected:



full_name	Overall Performance	Days Since Last Activity	Performance Level	Performance Diagnosis	Completion (Detail)	Reps Accuracy (Detail)	Last Activity (Detail)
user10	57.29	11	Needs Attention	low effort (sets complete, reps low)			
user11	28.94	11	Critical	critical (both sets and reps low)			
user12	43.07	10	Needs Attention	low effort (sets complete, reps low)			
user13	42.92	7	Needs Attention	low effort (sets complete, reps low)			
user14	50.00	10	Needs Attention	low effort (sets complete, reps low)			
user15	41.67	4	Needs Attention	low effort (sets complete, reps low)			
user16	50.00	3	Needs Attention	low effort (sets complete, reps low)			
user17	50.00	3	Needs Attention	low effort (sets complete, reps low)			
user19	77.44	0	Average	low effort (sets complete, reps low)			
user20	77.97	0	Average	low effort (sets complete, reps low)			
user21	102.90	0	Excellent	excellent (complete and accurate)			
user22	0.00	0	Critical	critical (both sets and reps low)			
user23	0.00	0	Critical	critical (both sets and reps low)			
user24	0.00	0	Critical	critical (both sets and reps low)			
user6	53.14	4	Needs Attention	critical (both sets and reps low)			
user7	44.52	3	Needs Attention	critical (both sets and reps low)			
user8	48.56	2	Needs Attention	low effort (sets complete, reps low)			
user9	46.07	6	Needs Attention	low effort (sets complete, reps low)			
Total	**52.62**	**5**		**low effort (sets complete, reps low)**			

When a single athlete has been selected:



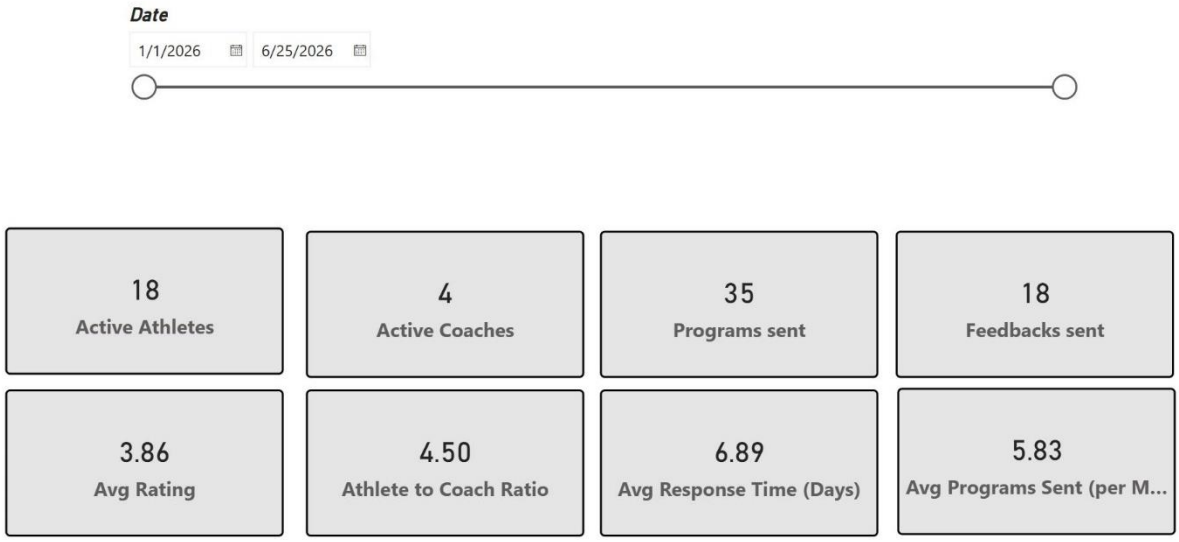
full_name	Overall Performance	Days Since Last Activity	Performance Level	Performance Diagnosis	Completion (Detail)	Reps Accuracy (Detail)	Last Activity (Detail)
user24	0.00	0	Critical	critical (both sets and reps low)		0.00	7/12/2026 12:00:00 AM
Total	**0.00**	**0**		**critical (both sets and reps low)**		**0.00**	**7/12/2026 12:00:00 AM**

3. Club Status

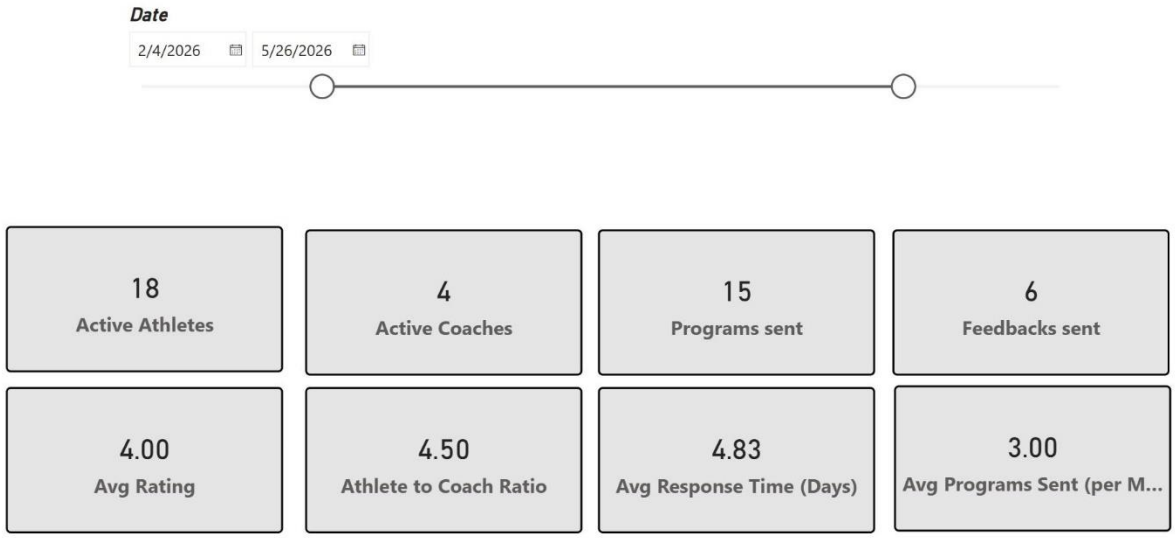
For this report, we use Cards because the report only needs six fields and, using Cards, the overall status can be seen without any complexity.

We also use a Slicer to specify the time period.

Overall report:



Report in a specified time period:



4. At-Risk Athlete Identification

This report helps identify athletes who are at risk of performance decline or dropout.

Using a scatter plot, the reason for each athlete's decline is visually identified (by looking at their position).

The horizontal axis shows the number of days since the athlete's last activity – moving to the right indicates a longer absence. The vertical axis displays the overall performance index – the lower a point is on the chart, the weaker the athlete's performance.

The chart is divided into four distinct quadrants by two lines:

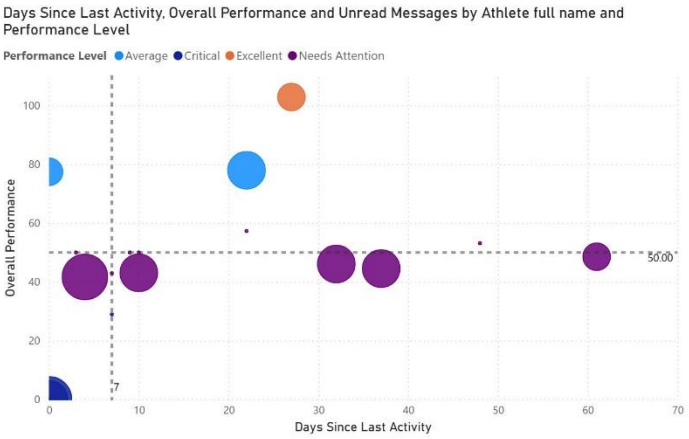
- A vertical line at day 7 (absence threshold).
- A horizontal line at 50% performance.

Quadrants:

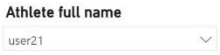
- Top-Left: Excellent performance and consistent activity – desirable status.
- Top-Right: Good performance but they haven't trained for a while – they need reminders and follow-ups.
- Bottom-Left: Low performance but still active – their training program needs adjustment.
- Bottom-Right: Very poor performance and they haven't been present at the gym for a long time – critical intervention needed.

Overall Report:

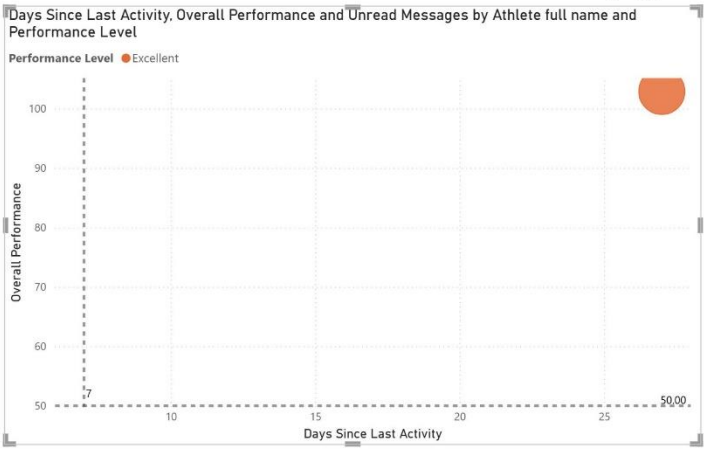
Athlete full name	Overall Performance	Days Since Last Activity	Unread Messages
user10	57.29	22	0
user11	28.94	7	0
user12	43.07	10	2
user13	42.92	7	0
user14	50.00	10	0
user15	41.67	4	3
user16	50.00	3	0
user17	50.00	9	0
user19	77.44	0	1
user20	77.97	22	2
user21	102.90	27	1
user22	0.00	0	2
user23	0.00	0	0
user24	0.00	0	3
user6	53.14	48	0
user7	44.52	37	2
user8	48.56	61	1
user9	46.07	32	2
Total	**52.62**	**17**	**19**



Report for a specific athlete:



Athlete full name	Overall Performance	Days Since Last Activity	Unread Messages
user21	102.90	27	1
Total	**102.90**	**27**	**1**



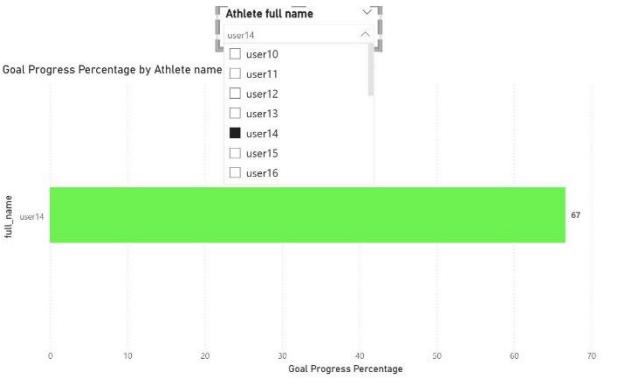
5. Athlete Goal Achievement

In this report, we used a line chart to display the progress trend of each athlete over time, and for comparing the progress of all athletes, we used a clustered bar chart.

Overall report:

Athlete name	goal_type	initial_value	current_value	target_value	Goal Progress Percentage	Estimated Time to Goal (Days)
user13	Weight Loss	0	0.00	0	0.00	
user8	Weight Loss	0	0.00	0	0.00	
user11	Weight Loss	65	58.00	50	46.67	19.43
user17	Strength Gain	55	63.00	75	40.00	7.50
user19	Weight Loss	75	65.00	60	66.67	1.00
user24	Strength Gain	50	65.00	80	50.00	2.00
user7	Strength Gain	50	65.00	80	50.00	6.00
user9	Weight Loss	78	70.00	60	44.44	7.50
user15	Strength Gain	65	72.00	85	35.00	7.43
user21	Weight Loss	80	72.00	65	53.33	1.75
user10	Strength Gain	60	75.00	90	50.00	13.00
user14	Weight Loss	85	75.00	70	66.67	5.00
user22	Strength Gain	60	75.00	85	60.00	1.33
user16	Weight Loss	90	85.00	75	33.33	10.00
user23	Weight Loss	95	85.00	75	50.00	2.00
user6	Weight Loss	92	85.50	75	38.24	11.31
user12	Strength Gain	80	88.00	95	53.33	21.00
user20	Strength Gain	100	95.00	110	-50.00	
Total					**27.00**	

Report for a specific athlete:



Athlete name	goal_type	initial_value	current_value	target_value	Goal Progress Percentage	Estimated Time to Goal (Days)
user14	Weight Loss	85	75.00	70	66.67	5.00
Total					**66.67**	**5.00**